

Response to Cui (2026): Incorporating the Analytical Solution into NIRApост for Ising Network Interventions

Fei Wang^{1,2}, Yiming Wu³, Yibo Wu⁴, Tingshao Zhu^{1,2}

1 State Key Laboratory of Cognitive Science and Mental Health, Institute of Psychology, Chinese Academy of Sciences, Beijing 100101, China

2 Department of Psychology, University of Chinese Academy of Sciences, 100049, Beijing, China

3 School of Psychology, South China Normal University, Guangzhou, China

4 Department of Nursing, the Fourth Affiliated Hospital of School of Medicine, and International School of Medicine, International Institutes of Medicine, Zhejiang University, Yiwu City, Zhejiang Province, China

1 Introduction

Cui (2026) noted that intervention outcomes in Ising models can be exactly obtained by enumerating all 2^p microscopic states, thereby eliminating Monte Carlo error—a proposition that holds mathematically. Our initial adoption of Monte Carlo simulation was motivated by two primary considerations. First, the number of states grows exponentially with the number of nodes (2^p), rendering complete enumeration computationally infeasible in both time and memory for large-scale networks, whereas Monte Carlo simulation offers a general-purpose approach scalable to networks of arbitrary size. Second, simulation-based frameworks remain the predominant methodology in current research. Nevertheless, Monte Carlo methods inevitably introduce stochastic error, and the repeated simulation stability checks we implemented to enhance result robustness have further increased computational costs. Given that analytical solutions offer both precision and efficiency for small- to medium-scale networks, we contend that incorporating analytical solutions into NIRApост as a complement to the Monte Carlo workflow is appropriate, allowing researchers to flexibly choose between approaches based on network size and specific requirements.

Accordingly, we provide two workflows in the updated *NIRApост* R package. The first is a Monte Carlo simulation-based workflow (comprising *simulateResponses*, *permutationNIRAtest*, and *stabilityNIRAtest*), intended for scenarios where network size precludes complete enumeration; it characterizes Monte Carlo error through simulation and repeated simulation. The second is an analytical solution-based workflow (*analyticalNIRAtest*), which exactly enumerates all 2^p microscopic states to compute intervention outcomes and quantifies sampling error via bootstrap resampling of the original data. Both workflows share the moderation effect testing procedure (*runMgmmAnalysis*) as a prerequisite.

2.1 The Ising Model and Exact Enumeration of Microstates

Consider an Ising model consisting of N nodes. The state of node i is described by a spin variable $v_i \in \{-1, +1\}$. The system's Hamiltonian is given by:

$$H(v) = -\frac{1}{2} \sum_{i=1}^N \sum_{j \neq i}^N \omega_{ij} v_i v_j - \sum_{i=1}^N \theta_i v_i \quad (1)$$

Where ω_{ij} denotes the coupling weight between nodes i and j , and θ_i represents the external field (threshold) of node i . According to the Boltzmann distribution, the probability of a state v is $P(v) = e^{-H(v)} / Z$, where the partition function is $Z = \sum_v e^{-H(v)}$. For a system with N nodes, there are 2^N possible microscopic states. In this study, we adopt an exact enumeration strategy and construct a state matrix $S \in \{-1, +1\}^{2^N \times N}$, where each row corresponds to a distinct microscopic state.

When model parameters are given in 0/1 coding, they must be converted to spin coding to ensure equivalence of probability distributions under both representations. The transformation formulas are as follows, where T_i is the threshold and W_{ij} is the weight:

$$\theta_i = \frac{1}{2} T_i + \frac{1}{4} \sum_{j \neq i} W_{ij}, \quad \omega_{ij} = \frac{1}{4} W_{ij} \quad (2)$$

Using Equations (1) and (2), we can compute the probability of every microscopic state for a given Ising network. However, meaningful interpretations typically concern macroscopic statistics of the system, such as the average activation level. We therefore define the number of active nodes as $n(v) = \sum_{i=1}^N I(v_i = +1)$. For the original system (without perturbation), the mean and standard deviation of the number of active nodes are:

$$\mu_0 = \frac{\sum_v n(v) f(v)}{Z}, \quad \sigma_0 = \sqrt{\frac{\sum_v n(v)^2 f(v)}{Z} - \mu_0^2} \quad (3)$$

Where $f(v) = e^{-H(v)}$ is the unnormalized weight and $Z = \sum_v f(v)$.

Combining Equations (1), (2), and (3) yields the average activation probability for a given Ising network with specified weight matrix and threshold vector. Analogous to NIRA, one could obtain analytical solutions by simulating perturbations of each node's intercept and repeating the above computations. However, repeated enumeration incurs substantial computational cost. To address this, we adopt a reweighting approach for node perturbations that requires only the computation of a correction factor for each perturbed node, eliminating the need for re-enumeration of original weights and recomputation of Hamiltonians. The derivation is outlined below.

Applying a threshold perturbation to node i such that $\theta_i \rightarrow \theta_i + \delta_i$, the perturbed Hamiltonian follows from Equation (1):

$$H^{pert}(v) = H(v) - \delta_i v_i \quad (4)$$

The unnormalized weight of the perturbed state becomes:

$$f^{pert}(v) = e^{-H^{pert}(v)} = f(v) \cdot e^{\delta_i v_i} \quad (5)$$

Equation (5) demonstrates that the perturbation effect is equivalent to multiplying the original weight by a correction factor that depends only on the spin value of the perturbed node. Consequently, the perturbed system's mean number of active nodes can be expressed as:

$$\mu_i = \frac{\sum_v n(v) f(v) \cdot e^{\delta_i v_i}}{\sum_v f(v) \cdot e^{\delta_i v_i}} \quad (6)$$

This is the reweighting formula. Its computation requires only a single enumeration of the original weights; for each perturbed node, only the correction factor $e^{\delta_i v_i}$ needs to be calculated,

without recomputing Hamiltonians. This approach avoids the computational overhead of repeated calculations, where p denotes the number of perturbed nodes.

2.2 Uncertainty Quantification: Bootstrap and Permutation Tests on the Original Data

Under fixed network parameters, the analytical solution is deterministic, and its uncertainty no longer arises from Monte Carlo simulation but rather from sampling error in the network parameters due to finite samples. Accordingly, *analyticalNIRAtest()* resamples the original data—re-estimating the network parameters and recomputing the analytical solution for each resample—to quantify this uncertainty, and delivers the following two sets of results within a single resampling loop:

(1) Bootstrap test (stored in *\$stat*): The original data rows are resampled with replacement *nResample* times, yielding standard errors, 95% confidence intervals, and p-values (testing $H_0: \Delta_i = 0$) for each node's intervention effect Δ_i , as well as the frequency with which each node ranks first across bootstrap replications (*percenttop_1*), serving as a stability metric within the analytical framework.

(2) Random-target test (stored in *\$random*): Building upon the resampled original data, the intervention targets are permuted across nodes to test whether node i 's intervention effect is significantly greater than that of a randomly selected target node; these p-values are not adjusted for multiple comparisons. The number of permutations is *nResample* $\times(p-1)$.

Distinction from the Monte Carlo-based permutation test: The original workflow (*permutationNIRAtest*) permutes group labels between the simulated original and intervention samples, thereby characterizing Monte Carlo error. In contrast, the analytical workflow eliminates Monte Carlo error; thus, the resampling target shifts from simulated samples to the original observed data, now characterizing sampling error. This aligns with the recommendation of Cui et al. (2026).

2.3 Visualization

The *analyticalNIRAtest()* function returns a *ggplot* object that displays the exact mean activation counts for each condition (the original network and each intervention node), accompanied by bootstrap 95% confidence interval error bars and significance asterisks based on multiplicity-adjusted p-values. Nodes are ordered by the direction of their effects, facilitating direct comparison with the *plotMeanNIRA()* figure from the original workflow.

3 Tutorial

analyticalNIRAtest() is the core function of the analytical-solution workflow. It estimates (or directly takes as given) the Ising network parameters from the original binary data, enumerates all 2^p microstates to precisely compute the mean activation counts for each node following intervention, and quantifies sampling uncertainty via resampling of the original data. The main parameters include: *data*, the original dataset; *perturbation_type*, the intervention direction (aggravating, alleviating); *edge_weights* and *thresholds*, optional network parameters (automatically estimated if omitted); *amount_of_SDs_perturbation*, the perturbation magnitude (default = 2, in units of the threshold standard deviation); *nResample*, the number of resampling iterations (default = 5000); *method*, the multiple-comparison correction procedure; and *seed*, *parallel*, and *ncores* for controlling the random seed and parallel computing.

```

res <- analyticalNIRAtest(
  data = single_gds,
  perturbation_type = "aggravating",
  edge_weights = edgeWeightMatrix,
  thresholds = thresholdVector,
  amount_of_SDs_perturbation = 2,
  nResample = 5000,
  method = "bonferroni",
  seed = 2025,
  parallel = TRUE,
  ncores = 6
)

```

The function returns a list: **\$stat** contains the bootstrap test results, providing for each node the intervention effect (**effect**: difference in mean activation between the post-intervention and original states), the standardized effect (**cohen_d**), standard errors and 95% confidence intervals, p-values (testing $H_0: \Delta = 0$) along with the adjusted **p.adjust**, and the frequency of ranking first across bootstrap replications (**percenttop_1**); **\$random** contains the random-target test results, providing uncorrected p-values for each node's effect being superior to that of a randomly selected target; **\$exact** provides the exact mean activation counts for each condition (the original network and each intervention node); and **\$plot** provides the resulting figure.

res\$stat

Table1 Analytical intervention effects and test results for each node (aggravating, 2 SD)

node	mean_activation	ci_lower	ci_upper	effect	cohen_d	se_effect	ci_effect_lower	ci_effect_upper	p	p.adjust	percenttop_1	stars
NA	4.62	4.49	4.80	1.24	0.44	0.03	1.20	1.32	0.00	0.00	0.00	**
UW	4.88	4.76	5.06	1.50	0.54	0.03	1.46	1.58	0.00	0.00	0.01	**
WTM	4.71	4.58	4.89	1.33	0.47	0.03	1.29	1.41	0.00	0.00	0.00	**
TR	4.81	4.68	4.98	1.43	0.51	0.03	1.38	1.51	0.00	0.00	0.00	**
RES	4.89	4.76	5.07	1.52	0.54	0.03	1.48	1.59	0.00	0.00	0.04	**
IRR	4.73	4.60	4.91	1.35	0.48	0.03	1.31	1.43	0.00	0.00	0.00	**
ASH	4.93	4.79	5.12	1.55	0.55	0.02	1.52	1.62	0.00	0.00	0.95	**

res\$random

Table 2 Random-Target Test Results (Aggravating, 2 SD)

node	effect	p
NA	1.24	1.00
UW	1.50	0.31
WTM	1.33	0.82
TR	1.43	0.51
RES	1.52	0.27

IRR	1.35	0.75
ASH	1.55	0.08

res\$plot

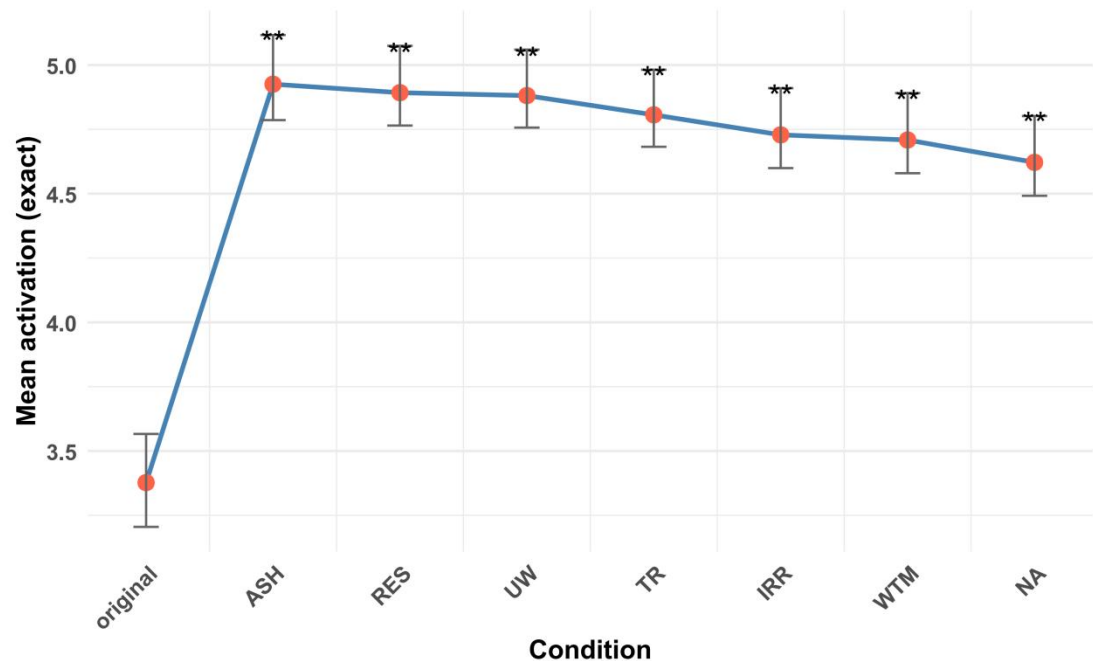


Figure 1 Significant exact effect diagram of simulated network intervention

Table 1, Table 2, and Figure 1 summarize the bootstrap test results, random-target test results, and the exact mean activation counts for each condition, respectively. Under the aggravating intervention on the GAD-7 network, all nodes exhibited significantly positive intervention effects (adjusted $p < 0.01$), with ASH showing the largest effect ($\Delta = 1.548$, 95% CI [1.519, 1.617]). ASH ranked first in 94.7% of the bootstrap replications; however, the p value from the random-target test was marginally significant ($p = 0.081$), indicating that ASH did not significantly outperform a randomly selected target at the 0.05 level. Nevertheless, this suggests that ASH remains the most robust and effective preferred intervention target.

Reference

Cui, J., Lunansky, G., Lichtwarck-Aschoff, A., Mendoza, N., & Hasselman, F. (2026). Quantifying the stability landscapes of psychological networks. *Behavior Research Methods*, 58, 68. <https://doi.org/10.3758/s13428-025-02917-7>

Henry, T. R., Robinaugh, D. J., & Fried, E. I. (2021). On the Control of Psychological Networks. *Psychometrika*. <https://doi.org/10.1007/s11336-021-09796-9>

Lunansky, G., Naberman, J., van Borkulo, C. D., Chen, C., Wang, L., & Borsboom, D. (2022). Intervening on psychopathology networks: Evaluating intervention targets through simulations. *Methods*, 204, 29–37. <https://doi.org/10.1016/j.ymeth.2021.11.006>

Newman, M. E. J., & Barkema, G. T. (1999, February). *Monte Carlo methods in statistical physics*. Oxford University Press. <https://doi.org/10.1093/oso/9780198517962.001.0001>

- Stocker, J. E., Koppe, G., Reich, H., Heshmati, S., Kittel-Schneider, S., Hofmann, S. G., Hahn, T., van der Maas, H. L. J., Waldorp, L., & Jamalabadi, H. (2023). Formalizing psychological interventions through network control theory. *Scientific Reports*, 13 (1), 13830. <https://doi.org/10.1038/s41598-023-40648-x>
- Wang, F., Wu, Y., Wu, Y., & Zhu, T. (2026). Simulation intervention for cross-sectional network models: Based on the R packages NodeIdentifyR and nirapost. <https://doi.org/10.1177/25152459261452944>